

fiuba

algo3

Temas varios de POO

(profundización)

Pablo Suárez
psuarez@fi.uba.ar

Contexto

Vemos que Pharo nos permite definir métodos y atributos “class side”

Pero no sabemos para qué sirven

Venimos usando UML desde el principio

Pero nos falta formalizar

Trabajamos con excepciones desde que vimos contratos

Pero no profundizamos en el tema en sí

Las clases se agrupan en paquetes

Pero no sabemos cuál es su semántica

¿Depende del lenguaje?

Temas varios de POO



Temario

Atributos y métodos de la clase

Inicializando objetos

Excepciones

Chequeo estático de excepciones

Diagrama de estados

Cierre de UML

Colecciones e iteradores

Genericidad

A modo de cierre de P00



Inicialización en Java

Hay constructores

```
Date d = new Date();
```

El constructor puede tener parámetros

```
Cuenta c = new CajaAhorro(1234, "Ana");
```

Debería dejar al objeto en un estado válido

=> debe cumplir con los invariantes

Si no hay constructor en una clase, el compilador crea uno por defecto

Ojo: sin parámetros

Inicialización en Smalltalk (1)

No hay constructores

sólo está el método de clase *new*

Debería dejar al objeto en un estado válido

=> debe cumplir con los invariantes

El *new* no es seguro

Hay un método *initialize*, que se puede redefinir

De todas maneras, no tiene parámetros

=> Tampoco es seguro

Inicialización en Smalltalk (2)

Por eso definimos un método:

```
CuentaBancaria >>  
  inicializarConNumero: numero conTitular: titular
```

Se invoca

```
cuenta := CuentaBancaria new  
  inicializarConNumero: 1234 conTitular: 'Juan'
```



¿Y si el número de cuenta tuviera que ser incremental?

Solución:

Agregar un atributo *ultimoNumero* que mantenga un único valor para la clase y todas sus instancias

Atributos de clase

En Smalltalk se declaran en “classVariableNames”

En Java se declaran *static*

CuentaBancaria
- <u>ultimoNumero : SmallInteger</u> - numero : SmallInteger - titular : String - saldo : SmallInteger
+ inicializar(titular : String) + getSaldo() : SmallInteger + depositar(monto : SmallInteger) + extraer(monto : SmallInteger) + getTitular() : String + <u>getProximoNumero() : SmallInteger</u> + getNumero() : SmallInteger

A modo de repaso: implementación

Escribimos prueba

```
PruebaCuentaBancaria >> testAutoIncremental
```

```
  cuenta1 := CuentaBancaria new
```

```
    inicializarConTitular: 'Juan Pérez'.
```

```
  cuenta2 := CuentaBancaria new
```

```
    inicializarConTitular: 'Ana García'.
```

```
  assert: ( (cuenta2 getNumero) - (cuenta1 getNumero) = 1)
```

Nos aseguramos de que no pase

Implementamos la solución

Nos aseguramos de que corra

Mensajes enviados a la clase

¿Qué hicimos cuando escribimos ... ?

lista := CuentaCorriente new.

CuentaCorriente es una clase

Métodos de clase

En Smalltalk, las clases son objetos

En Java, se declaran *static*

Inicialización en Smalltalk (3)

Con método de clase:

```
CuentaBancaria >>  
  inicializarConNumero: numero conTitular: titular
```

Se invoca

```
cuenta := CuentaBancaria  
  inicializarConNumero: 1234 conTitular: 'Juan'
```



Recapitulación: preguntas

¿Para qué sirve un atributo de clase?

¿Y un método de clase?

¿Cómo inicializamos objetos en Smalltalk?



Más sobre excepciones

Error al copiar un archivo o carpeta



No se puede copiar court.jpg: El nombre de archivo especificado no es válido o es demasiado largo. Especifique otro nombre de archivo.

Aceptar

Cuándo lanzar excepciones

Si en el contexto en el que estamos no hay suficiente información para resolver el potencial problema

Hablamos de excepciones cuando el problema no se puede resolver en un determinado contexto

Y la lanzamos a un contexto de nivel superior para que resuelva qué hacer

En el modelo contractual, una excepción ocurre cuando no se cumple una precondition

También para aislar el código que se usa para tratar problemas del código básico (camino feliz)

Para crear software más robusto

Qué hacer al capturar

Resolverla mediante

- Finalización súbita

- Continuación ignorando las fallas

- Avance y recuperación

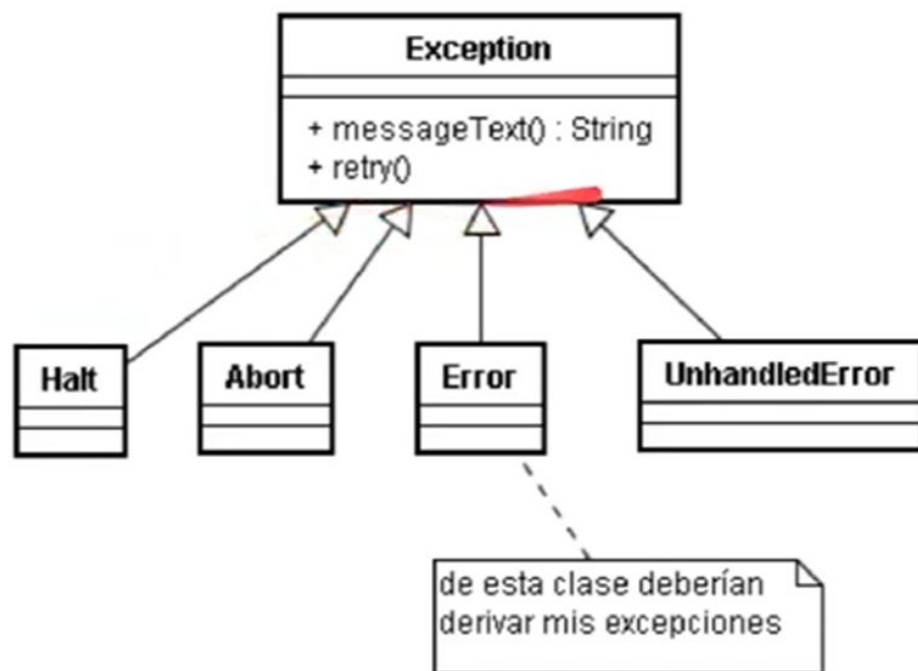
- Nuevo intento

No resolverla y enviarla al contexto invocante

- La misma

- Otra excepción

Smalltalk: jerarquía de excepciones



La jerarquía influye en la captura

Cuando decimos capturar un tipo de excepción, capturamos cualquier instancia de esa clase o una descendiente

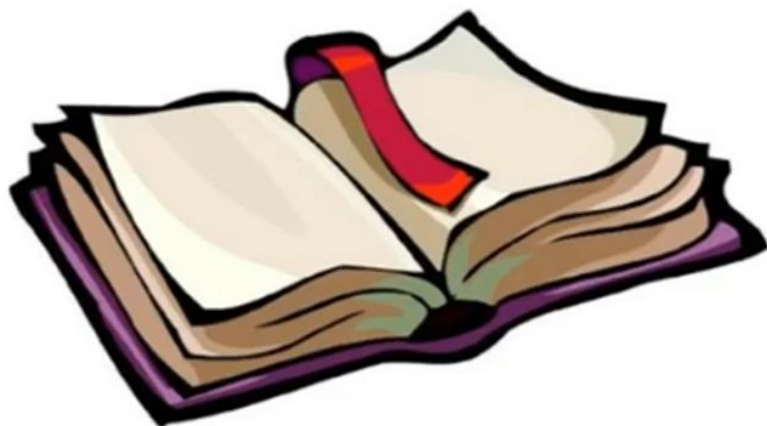
Jerarquías de excepciones propias

Sirven para dar mayor información sobre el
tipo de problema

Podrían agregar atributos y métodos

Pero no es lo más habitual

Cuidar bien la jerarquía

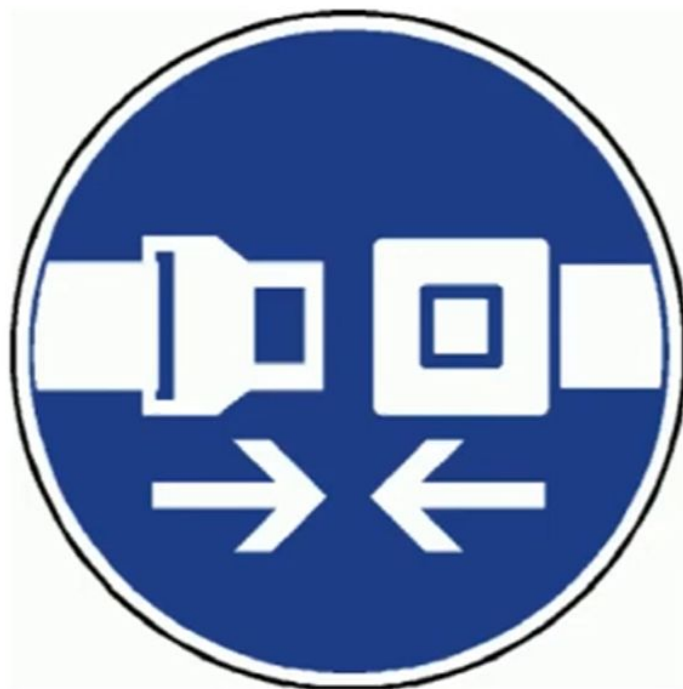


Recapitulación: preguntas

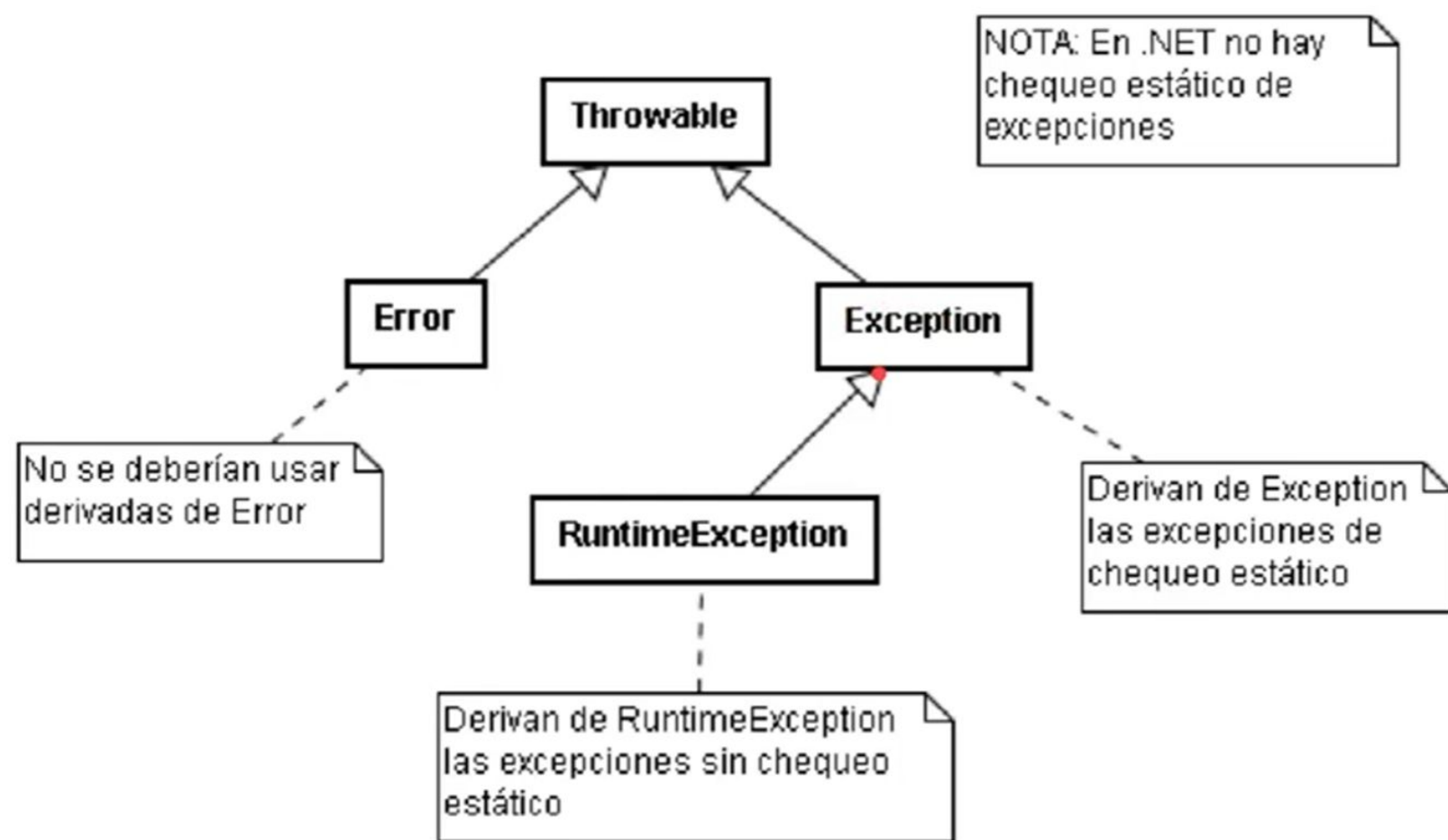
- ¿Por qué deberíamos crear nuestras clases de excepción?
- ¿En qué casos volvemos a lanzar una excepción al capturar?



Excepciones chequeadas en forma estática (Java)



Jerarquía de excepciones (Java)



Excepciones chequeadas

Cláusula “throws” obligatoria

```
public Fraccion dividir (Fraccion y) throws FraccionInvalidaException {  
    if (y.numerador == 0)  
        throw new FraccionInvalidaException ( );  
    int numerador = this.numerador * y.denominador;  
    int denominador = this.denominador * y.numerador;  
    return new Fraccion(numerador, denominador);  
}
```

A lo sumo se puede declarar un ancestro

En redefiniciones, mantener y no agregar

Para mantener el polimorfismo: muy molesto

Obligación de capturar: chequeada por el compilador

Posible captura (1)

```
public Fraccion divisionMultiple ( Fraccion [ ] x, Fraccion [ ] y ) {  
    Fraccion suma = new Fraccion (0, 1);  
    try {  
        for (int i = 0; i < 10; i++) {  
            Fraccion d = x[i].dividir ( y [i] );  
            suma = suma.sumar(d);  
        }  
    } catch (FraccionInvalidaException e) {  
        tratamientoExcepcion(e);  
    }  
    return suma;  
}
```



Posible captura (2)

```
public Fraccion divisionMultiple ( Fraccion [ ] x, Fraccion [ ] y )  
    throws FraccionInvalidaException {  
    Fraccion suma = new Fraccion (0, 1);  
    for (int i = 0; i < 10; i++) {  
        Fraccion d = x[i].dividir( y[i] );  
        suma = suma.Sumar(d);  
    }  
    return suma;  
}
```



Posible captura (3)

```
public Fraccion divisionMultiple ( Fraccion [ ] x, Fraccion [ ] y) {  
    Fraccion suma = new Fraccion (0, 1);  
    try {  
        for (int i = 0; i < 10; i++) {  
            Fraccion d = x[i].dividir( y[i] );  
            suma = suma.sumar(d);  
        }  
    } catch (FraccionInvalidaException e) { }  
    return suma;  
}
```



Lenguajes y excepciones

Excepciones chequeadas

- Son más seguras

- Molesta tener que capturarlas sí o sí

- Limita la redefinición, al no poder agregar nuevas excepciones

- Se hizo para cumplir con el principio de substitución

Microsoft diseñó .NET sin excepciones chequeadas

Ojo: Java permite ambas

- Aunque es una decisión de diseño

Recapitulación: preguntas

¿Qué significa exactamente que Java tiene un mecanismo de excepciones chequeadas en tiempo de compilación?

¿Qué deberíamos hacer si no deseamos usarlo?

